

CS-405: Deep Learning

Transformer: Attention is All You Need!

Dr. Sajjad Hussain

SEEDS, NUST

April 21st, 2026

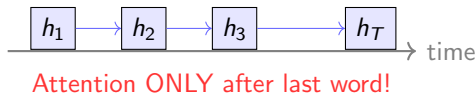
Lecture Outline

- 1 Self Attention
- 2 Multi-Head Attention
- 3 Transformer
 - Encoder
 - Decoder

Limitations of RNN Attention

RNN Attention Must Wait

In RNN-based encoder-decoder, attention can only start **AFTER** the encoder processes the entire input.



Key Problems:

- Cannot compute attention in parallel
- Long-range dependencies still difficult
- Each decoder step re-computes attention from scratch

Self-Attention: Attend to ALL words SIMULTANEOUSLY!

Self-Attention: The Core Intuition

Key Idea

Each word looks at **ALL other words in the sentence** to understand itself better.

Example: French sentence

"Le garçon jouait dans le parc hier soir"

For the word "jouait" (was playing):

- Look at "garçon" (who?)
- Look at "parc" (where?)
- Look at "hier soir" (when?)

For the word "parc" (park):

- Look at "jouait" (action?)
- Look at "dans" (preposition?)
- Look at "le" (article?)

Every word becomes a "query" looking for relevant "keys"!

The Query, Key, Value Analogy

Library Search Analogy

You (Query)	Book Labels (Keys)	Book Content (Values)
"I need books about History"	"Fiction book" → low match "History book" → high match "Culture book" → medium match	Actual information from matching books

Role	In Library	In Self-Attention
Query (Q)	What you're looking for	"What am I interested in?"
Key (K)	What each book says about itself	"What do I offer?"
Value (V)	Actual content of the book	"My actual meaning"

Each word creates its own Q, K, V through learned matrices!

Self-Attention for One Word: "parc"

Our Sentence

French: *"Le garçon jouait dans le parc hier soir"*

English Translation: *"The boy was playing in the park yesterday evening."*

Goal: Compute attention for "parc" (park)



Example

Step 1: Create Query for "parc"

$$q_6 = W^Q \cdot \text{embedding}(\text{"parc"})$$

This query asks: "What words are related to 'park'?"

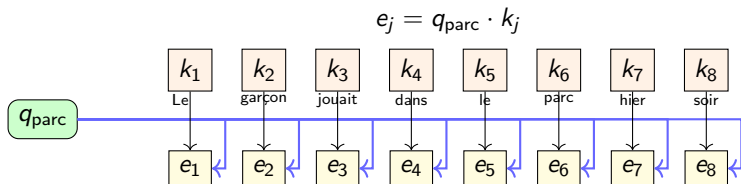
Step 2: Compare with all Keys

$$\text{score}_{6,j} = q_6 \cdot k_j \quad \text{for } j = 1, 2, \dots, 8$$

Step 3: Softmax to get attention weights

$$\alpha_{6,j} = \frac{\exp(\text{score}_{6,j})}{\sum_{m=1}^8 \exp(\text{score}_{6,m})}$$

Computing Attention Scores for "parc"



Expected Scores (Intuition):

Word:	Le	garçon	jouait	dans	le	parc	hier	soir
Score:	0.1	0.3	0.7	0.4	0.1	0.5	0.2	0.2

Highest score for "jouait" (playing) — the action in the park!

Softmax and Attention Weights for "parc"

Apply Softmax to Get Probabilities

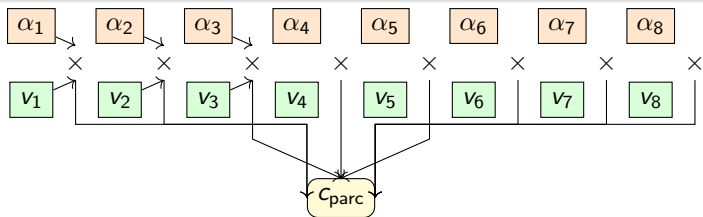
$$\alpha_j = \frac{e^{e_j}}{\sum_{m=1}^8 e^{e_m}}$$

Word	Attention Weight α
Le (the)	0.02
garçon (boy)	0.12
jouait (playing)	0.45 (highest)
dans (in)	0.10
le (the)	0.01
parc (itself)	0.18
hier (yesterday)	0.07
soir (evening)	0.05

Computing Weighted Sum (Context Vector)

Weighted Sum of Values

$$\text{attention_output}_{\text{parc}} = \sum_{j=1}^8 \alpha_j \cdot v_j$$



$$c_{\text{parc}} = 0.02v_1 + 0.12v_2 + 0.45v_3 + \dots$$

Result: The new representation for "parc" contains information from:

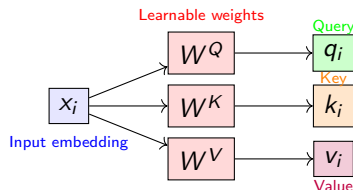
- "jouait" (playing) — the action happening here
- "garçon" (boy) — who is playing
- "hier soir" (yesterday evening) — when it happened

The Q, K, V Matrices: Learning to Attend

Each word has three vectors!

From each input embedding x_i , we compute:

$$q_i = W^Q x_i, \quad k_i = W^K x_i, \quad v_i = W^V x_i$$



Why Separate Matrices?

- W^Q , W^K , W^V are **learned during training**
- Different roles allow different patterns to be captured
- Same input x_i can be transformed for different purposes

The Complete Self-Attention Formula

Self-Attention in One Equation

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Where:

- Q : Matrix of all Queries ($T \times d_k$)
- K : Matrix of all Keys ($T \times d_k$)
- V : Matrix of all Values ($T \times d_v$)
- T : Sequence length
- d_k : Dimension of keys/queries
- d_v : Dimension of values

Step-by-step:

- 1 QK^T : Compare every query with every key
- 2 $\frac{1}{\sqrt{d_k}}$: Scale to prevent large values
- 3 softmax: Convert to probabilities
- 4 $\times V$: Weighted sum of values

$$Q \times K^T \rightarrow \text{scale} \rightarrow \text{softmax} \rightarrow \times V \rightarrow \text{Output}$$

Why Scale by $\sqrt{d_k}$?

The Problem

For large d_k , dot products become very large $\rightarrow$ softmax becomes very peaked (one value near 1, others near 0).

Example with $d_k = 512$:

- Each q_i and k_j have 512 dimensions
- Dot product values can be in range $[-50, 50]$
- $\exp(50)$ is enormous! Softmax becomes almost one-hot
- Gradients become very small (vanishing gradients)

The Fix: Divide by $\sqrt{d_k}$

$$\text{score} = \frac{q_i \cdot k_j}{\sqrt{d_k}}$$

Scaling keeps the variance constant, preventing overly peaked softmax!

Self-Attention for ALL Words in Parallel

Key Advantage of Self-Attention

All attention scores can be computed **SIMULTANEOUSLY** using matrix multiplication!

Computation:

- 1 $QK^T: [T \times d_k] \times [d_k \times T] = [T \times T]$
- 2 softmax + scale: still $[T \times T]$
- 3 $\times V: [T \times T] \times [T \times d_v] = [T \times d_v]$

$O(1)$ sequential operations — massively parallel!

Key Concepts:

- Each word attends to **ALL words**
- Query: "What am I looking for?"
- Key: "What do I offer?"
- Value: "My actual information"
- Parallel computation via matrix multiplication

Advantages over RNNs:

- No sequential bottleneck
- Direct connections between any positions
- No vanishing gradients
- Highly parallelizable
- Interpretable attention weights

Self-Attention Summary

For Our French Sentence Example

"Le garçon jouait dans le parc hier soir"

After self-attention:

- "parc" (park) knows about "jouait" (playing)
- "jouait" (playing) knows about "garçon" (boy) and "parc" (park)
- "garçon" (boy) knows about "jouait" (playing)

Next: Multi-Head Attention and The Transformer!

Why Single-Head Attention Isn't Enough

The Limitation of Single-Head Attention

One attention mechanism can only capture **ONE** type of relationship at a time.

Example: The word "parc" (park) in our sentence

"Le garçon jouait dans le parc hier soir"

A single attention head must answer ALL these questions simultaneously:

- "WHO is doing the action?" → garçon (boy)
- "WHAT action is happening?" → jouait (playing)
- "WHERE is the park?" → dans (in) — grammatical role
- "WHEN did it happen?" → hier soir (yesterday evening)

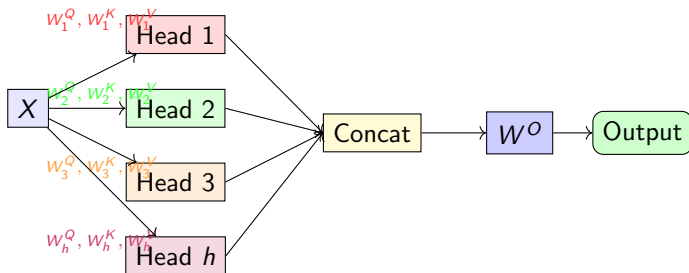
Single head blends ALL information together!

Multi-Head Attention: The Core Idea

Instead of one attention, use MULTIPLE in parallel!

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h) W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$



Each head learns DIFFERENT relationships independently!

Intuition: Different Heads for Different Relationships

Our Example: "Le garçon jouait dans le parc hier soir"

**Head 1: Subject
focus**

garçon gets high
weight

**Head 2: Action
focus**

jouait gets high
weight

**Head 3: Time
focus**

hier soir gets
high weight

**Head 4:
Grammar focus**

dans gets high
weight

Multi-Head for "parc": Detailed Computation

Each Head Has Its Own Q, K, V Matrices

For head i :

$$q_i = W_i^Q \cdot x_{\text{parc}}, \quad k_i^{(j)} = W_i^K \cdot x_j, \quad v_i^{(j)} = W_i^V \cdot x_j$$

Head 1 (Subject Focus):

Word:	Le	garçon	jouait	parc	...
Score:	0.05	0.85	0.05	0.03	...

$$\text{output}_1 = 0.05 \cdot v_1(\text{Le}) + 0.85 \cdot v_2(\text{garçon}) + \dots$$

Head 2 (Action Focus):

Word:	Le	garçon	jouait	parc	...
Score:	0.02	0.08	0.78	0.05	...

$$\text{output}_2 = 0.02 \cdot v_1(\text{Le}) + 0.08 \cdot v_2(\text{garçon}) + 0.78 \cdot v_3(\text{jouait}) + \dots$$

Multi-Head for "parc": Computations Cont'd

Head 3 (Time Focus):

Word:	...	parc	hier	soir
Score:	0.05	0.03	0.55	0.32

$$\text{output}_3 = \dots + 0.03 \cdot v_6(\text{parc}) + 0.55 \cdot v_7(\text{hier}) + 0.32 \cdot v_8(\text{soir})$$

The Complete Multi-Head Formula

Step 1: Compute each head

$$\text{head}_i = \text{Attention}(Q, K, V) = \text{softmax} \left(\frac{(QW_i^Q)(KW_i^K)^\top}{\sqrt{d_k}} \right) (VW_i^V)$$

where:

- $W_i^Q, W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$
- $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$
- $d_k = d_v = d_{\text{model}}/h$ (typically)

The Complete Multi-Head Formula

Step 2: Concatenate and Project

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h) W^O$$

where $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$

Example Dimensions (BERT-base):

- $d_{\text{model}} = 768$ (embedding dimension)
- $h = 12$ heads
- $d_k = d_v = 768/12 = 64$ per head

Mathematical Walkthrough with Numbers

Assumptions for Our Example

- $d_{\text{model}} = 4$ (tiny, for illustration)
- $h = 2$ heads (simplified)
- $d_k = d_v = 2$ per head

For Head 1 (Subject Focus):

$$W_1^Q, W_1^K \in \mathbb{R}^{4 \times 2}, \quad W_1^V \in \mathbb{R}^{4 \times 2}$$

For Head 2 (Action Focus):

$$W_2^Q, W_2^K \in \mathbb{R}^{4 \times 2}, \quad W_2^V \in \mathbb{R}^{4 \times 2}$$

Head	Learns to focus on	Typical attention pattern
1	Subject ("garçon")	$\alpha_{\text{garçon}} \approx 0.7$
2	Action ("jouait")	$\alpha_{\text{jouait}} \approx 0.6$

Concatenation and Final Output

After Computing All Heads

For the word "parc", each head produces a d_v -dimensional vector:

$$\text{head}_1(\text{parc}) = [0.2, 0.5] \quad (\text{subject info})$$

$$\text{head}_2(\text{parc}) = [0.7, 0.1] \quad (\text{action info})$$

$$\text{head}_3(\text{parc}) = [0.3, 0.4] \quad (\text{time info})$$

Concatenate all heads:

$$\text{Concat} = [0.2, 0.5, 0.7, 0.1, 0.3, 0.4] \quad (\text{length} = h \times d_v = 6)$$

Project back to d_{model} :

$$\text{Final} = \text{Concat} \cdot W^O \quad \text{where } W^O \in \mathbb{R}^{6 \times 4}$$

$$\text{Final} = [0.35, 0.42, 0.28, 0.51]$$

The final vector contains information from ALL perspectives!

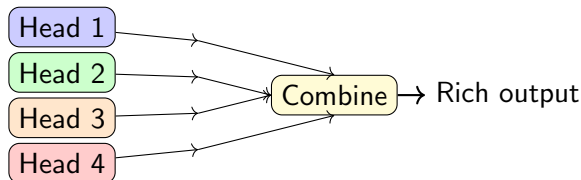
Why Multi-Head Works: The Intuition

Single Head:

- One weighted average
- Information gets **blurred together**
- "Park" receives mixed signals
- Hard to disentangle

Multi-Head:

- Multiple parallel representations
- Each head specializes
- "Park" gets separate signals:
 - Subject info
 - Action info
 - Time info
 - Grammatical info



Multi-Head Attention: Summary

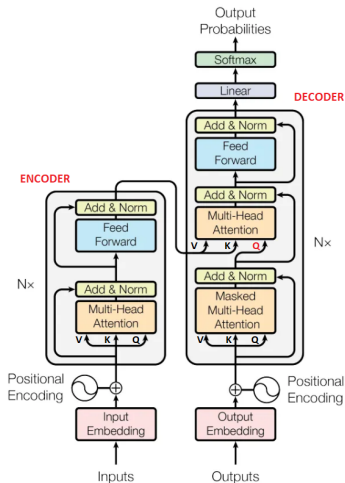
- ① **Parallel Attention:** h heads compute attention simultaneously
- ② **Different Projections:** Each head has its own W^Q, W^K, W^V matrices
- ③ **Specialization:** Different heads learn different linguistic patterns
 - Some focus on syntactic relationships
 - Some focus on semantic relationships
 - Some focus on long-range dependencies
 - Some focus on positional information
- ④ **Concatenation:** Outputs from all heads are combined
- ⑤ **Final Projection:** W^O mixes information across heads

Model	# Heads (h)	d_{model}
BERT-base	12	768
BERT-large	16	1024
GPT-3	96	12288
T5-base	12	768

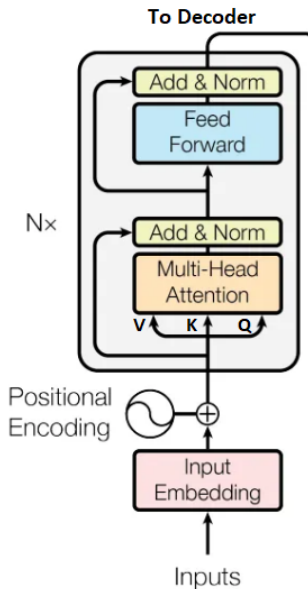
The Transformer: Big Picture

Transformers

- Transformer = **Attention is All You Need** (Vaswani et al., 2017)



Encoder Architecture



Encoder Architecture: Like a CNN for Text

Key Insight

Just as CNN extracts hierarchical features from images, the Transformer encoder extracts **rich contextual representations** from text.

For Images (CNN):

- Pixels $\rightarrow$ Edges $\rightarrow$ Shapes $\rightarrow$ Objects
- Hierarchical feature extraction
- Translation invariant

For Text (Transformer Encoder):

- Words $\rightarrow$ Local context $\rightarrow$ Global understanding
- Self-attention captures all relationships
- Positional encoding preserves order

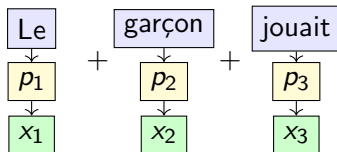
Positional Encoding: Adding Order Information

Self-attention has NO notion of word order!

Without positional info, "cat sat on mat" and "mat sat on cat" would be the same.

Solution: Add positional vectors to word embeddings

Final Input = Word Embedding + Positional Encoding



Positional Encoding Formula (Sinusoidal):

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

Different positions get UNIQUE encoding patterns!

Encoder Self-Attention: Every Word Sees Everything

In the Encoder, attention is UNMASKED (Bidirectional)

Each word can attend to **ALL words** in the input sentence.

For "jouait" (was playing):

- **Looks LEFT:** "Le", "garçon" (boy — who is playing)
- **Looks AT ITSELF:** "jouait"
- **Looks RIGHT:** "dans", "le", "parc" (park — where), "hier", "soir" (when)

No masking! Full bidirectional understanding!

Encoder Output: Deep Contextual Representations

After N Encoder Layers

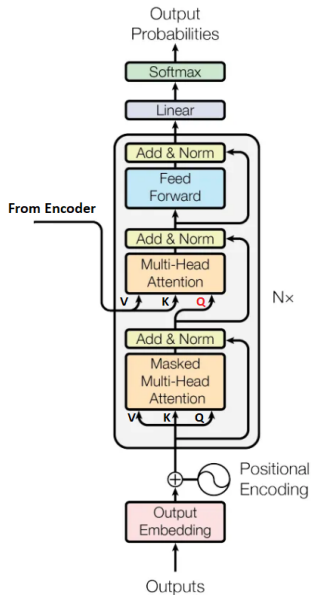
Each word's representation contains information from **ALL words**.

Word	What its final representation knows
Le	Grammatical role, attaches to "garçon"
garçon	Subject of "jouait", male, singular
jouait	Action by "garçon", in "parc", yesterday evening
dans	Preposition introducing "parc"
le	Article for "parc"
parc	Location of "jouait", modified by "dans"
hier	Time modifier of "jouait"
soir	Pairs with "hier" for complete time

These representations are passed to the Decoder!

- Encoder outputs become **KEYS and VALUES** for Decoder cross-attention

Decoder Architecture



The Decoder's Job: Generate One Word at a Time

The Big Picture

The decoder takes the encoder's deep representations of the Input sentence (e.g. French) and generates the English translation **WORD BY WORD**.

Key Principle: Autoregressive Generation

- The decoder starts with only the **<SOS>** (Start of Sequence) token
- It predicts the **FIRST** English word
- Then it uses **<SOS>** + the predicted word to predict the **SECOND** word
- This continues until the **<EOS>** (End of Sequence) token is generated

The decoder builds the translation progressively, word by word!

Step 1: Starting with $\langle \text{SOS} \rangle$

Decoder Input at Step 1

Input = $[\langle \text{SOS} \rangle]$

What Happens:

① Masked Self-Attention:

- Only one token ($\langle \text{SOS} \rangle$), so nothing to mask
- Produces Query Q_1 from $\langle \text{SOS} \rangle$

② Cross-Attention:

- Q_1 (from $\langle \text{SOS} \rangle$) attends to Encoder's K and V (French sentence)
- The decoder asks: "How do I start translating this French sentence?"
- Attention focuses strongly on "Le" (the) and context

③ Feed Forward + Softmax:

- Produces probability distribution over English vocabulary
- Highest probability: "The" (because "Le" means "the")

Step 1 Output: "The"

Step 2: Generating "boy"

Decoder Input at Step 2

Input = [$\langle$ SOS $\rangle$, "The"]

What Happens:

1 Masked Self-Attention:

- "The" can attend to $\langle$ SOS $\rangle$ (previous)
- $\langle$ SOS $\rangle$ cannot attend to "The" (future) $\rightarrow$ masked
- Produces Query Q_2 containing context of " $\langle$ SOS $\rangle$ The"

2 Cross-Attention:

- Q_2 attends to Encoder's K and V
- The decoder asks: "Given that I've said 'The', what comes next?"
- Attention focuses on "garçon" (boy)

3 Feed Forward + Softmax:

- Produces probability distribution
- Highest probability: "boy"

Step 2 Output: "boy"

Step 3: Generating "was playing"

Decoder Input at Step 3

Input = [$\langle$ SOS $\rangle$, "The", "boy"]

What Happens:

① Masked Self-Attention:

- "boy" can attend to $\langle$ SOS $\rangle$ and "The" (previous)
- Cannot attend to any future words (none exist)
- Produces Query Q_3 containing context of "The boy"

② Cross-Attention:

- Q_3 attends to Encoder's K and V
- The decoder asks: "What action does 'the boy' perform?"
- Attention focuses on "jouait" (was playing)

③ Feed Forward + Softmax:

- Produces probability distribution
- Highest probability: "was playing"

Step 3 Output: "was playing"

Step 9: Generating <EOS> (Stop!)

Decoder Input at Step 9

Input = [<SOS>, "The", "boy", "was playing", $\dots$ "yesterday", "evening"]

What Happens:

1 Masked Self-Attention:

- "evening" attends to all previous words
- Produces Query Q_9 with the complete English sentence

2 Cross-Attention:

- Q_9 looks at encoder — everything has been translated

3 Feed Forward + Softmax:

- The model recognizes the sentence is complete
- Highest probability: <EOS>

Step 9 Output: <EOS> $\rightarrow$ STOP GENERATION!

Complete Translation Summary

Step	Decoder Input	Output Generated
1	[<SOS>]	"The"
2	[<SOS>, The]	"boy"
3	[<SOS>, The, boy]	"was playing"
4	[<SOS>, The, boy, was playing]	"in"
5	[<SOS>, The, boy, was playing, in]	"the"
6	[<SOS>, The, boy, was playing, in, the]	"park"
7	[..., park]	"yesterday"
8	[..., yesterday]	"evening"
9	[..., evening]	<EOS>

Final Translation

"The boy was playing in the park yesterday evening"

Success! Every French word has been translated correctly!

Add & Norm: The Residual Connection

What is "Add"?

The "Add" refers to a **residual connection** (skip connection) that adds the input of a sub-layer to its output.

$$\text{Add}(x) = x + \text{Sublayer}(x)$$

Why Residual Connections?

- **Prevents Vanishing Gradients:** Gradients can flow directly through the skip connection, avoiding deep layers
- **Eases Training:** The model can learn identity mapping if the sub-layer doesn't need to transform the input
- **Enables Deeper Networks:** Transformers can have 100+ layers because residuals stabilize training

In the Transformer:

- After Multi-Head Self-Attention: $x \rightarrow x + \text{Attention}(x)$
- After Feed Forward Network: $x \rightarrow x + \text{FFN}(x)$

Add & Norm: Layer Normalization

What is "Norm" ?

"Norm" refers to **Layer Normalization** which normalizes the activations across the feature dimension.

$$\text{LayerNorm}(x) = \frac{x - \mu}{\sigma} \cdot \gamma + \beta$$

where μ and σ are mean and standard deviation across features.

Why Layer Normalization?

- **Stabilizes Training:** Keeps activations within a reasonable range
- **Faster Convergence:** Reduces internal covariate shift
- **Works Well for Sequences:** Normalizes each sample independently (unlike BatchNorm)

In the Transformer:

- Applied **AFTER** adding the residual connection
- Complete formula: $\text{LayerNorm}(x + \text{Sublayer}(x))$

Feed-Forward Network (FFN): Overview

What is the FFN?

A simple, position-wise fully connected network applied **independently** to each position in the sequence.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Key Properties:

- **Position-Wise:** Same network applied to EVERY token separately
- **Two Linear Layers:** Expands dimension, then contracts back
- **ReLU Activation:** Introduces non-linearity between layers

Typical Dimensions (BERT-base):

- Input dimension: $d_{\text{model}} = 768$
- Hidden dimension: $d_{\text{ff}} = 3072$ ($4\times$ larger)
- Output dimension: $d_{\text{model}} = 768$

FFN = "Position-wise" = Same network, different positions!

FFN: What It Does

Purpose of the Feed-Forward Network

The FFN adds **non-linearity and transformation capacity** to the model.

Why is FFN Needed?

- **Self-Attention is Linear:** Attention computes weighted sums (linear operations)
- **Need Non-Linearity:** Without FFN, stacking attention layers would still be linear
- **FFN Adds Expressive Power:** Enables complex pattern learning

Information Flow:

Token Representation $\xrightarrow{\text{Attention}}$ Contextual Info $\xrightarrow{\text{FFN}}$ Transformed Representation

FFN = "Processing" after "Mixing"!

FFN: Same in Encoder and Decoder

Identical FFN Structure in Both Encoder and Decoder

Both use the same position-wise feed-forward network with identical dimensions.

Component	Encoder FFN	Decoder FFN
Input dim	d_{model}	d_{model}
Hidden dim	$d_{\text{ff}} (4 \times d_{\text{model}})$	$d_{\text{ff}} (4 \times d_{\text{model}})$
Activation	ReLU	ReLU
Output dim	d_{model}	d_{model}
Position-wise	Yes	Yes

What FFN Learns:

- **In Encoder:** Transforms deep contextual representations of French words
- **In Decoder:** Transforms representations of English generation before output

Final Linear Layer & Softmax: Overview

What Comes After the Decoder?

The final decoder output passes through two layers to produce word predictions.

The Two Layers:

- 1 **Linear Layer:** Projects decoder output to vocabulary size

$$\text{logits} = h_{\text{decoder}} \cdot W_{\text{out}} + b_{\text{out}}$$

where $W_{\text{out}} \in \mathbb{R}^{d_{\text{model}} \times |V|}$

- 2 **Softmax:** Converts logits to probabilities

$$P(y_t | y_{<t}, x) = \frac{\exp(\text{logits}_t)}{\sum_{v=1}^{|V|} \exp(\text{logits}_v)}$$

Linear = Map to vocabulary; Softmax = Convert to probabilities!

Final Linear & Softmax: Training vs Inference

During TRAINING

- Full sequence available (teacher forcing)
- Softmax produces probability for EACH position
- Loss: Cross-entropy between predictions and true next token

$$\mathcal{L} = - \sum_{t=1}^T \log P(y_t^{\text{true}} | y_{<t}, x)$$

During INFERENCE (Generation)

- Only the **last position** matters
- Softmax produces probability for NEXT token only
- Select token via argmax or sampling

$$\hat{y}_t = \operatorname{argmax}_v P(v | y_{<t}, x)$$